

Formalizing SystemVerilog in lean-surfaces

An onboarding brief: strategy, current state, and the road to a verified RISC-V core

lean-surfaces project

July 2026 (repo: github.com/thomasnormal/lean-surfaces)

1 What the project is

lean-surfaces proves correctness of *real* programs — Python, SystemVerilog, and analog circuits (SPICE) — in Lean 4, via **deep embeddings**: each language’s own frontend (CPython `ast`, slang/pyslang, a native SPICE parser) dumps the program’s real AST into a standardized JSON envelope; Lean ingests it as a literal term; an *executable definitional interpreter* gives it meaning; and theorems are stated against that interpreter in a polished surface notation. Programs stay **source-shaped**: the Lean term mirrors the file the engineer wrote. This is the project’s binding design constraint, because the primary prover is an AI — *legibility of proof goals* decides everything. When a goal appears mid-proof it must read like the source program and its mathematics (`IDLE, count, 2 · total = i(i − 1)`), never like an elaborated intermediate representation.

Four **decoupled coverage axes** keep this honest: (1) *parse* coverage ($\approx 100\%$, borrowed frontends); (2) *representation* (full ASTs; unknown constructs become loud `Unsupported` nodes — ingestion never fails); (3) *semantics* (tiered interpreters that return `.unsupported` rather than guess; coverage measured on real corpora, never claimed); (4) *proofs* (the spec layer, which lags semantics by design). Two further principles matter daily. The **oracle principle**: all nondeterminism is an explicit oracle parameter — the SV scheduler’s same-region ordering freedom is a quantified σ , so theorems range over *all legal schedules*, and environment nondeterminism (e.g. bus response latency) will be a second oracle β with fairness bundled into its legality. And **differential testing as epistemology**: every semantic rule is validated against ground truth *before* theorems may use it — CPython for Python, *Xcelium* and *Icarus Verilog* for SV, ngspice/Spectre for analog, and the Sail emulator for the ISA model. Kernel proofs tell us what our semantics implies; differential harnesses tell us our semantics is faithful. Neither substitutes for the other.

2 The SV lane today

The lane lives in `LeanModels/Sv/**` with examples in `Examples/system-verilog/<name>/` (three-file layout: `<name>.sv`, extracted `.json`, `spec.lean` with statements proved `:= by proofs`, `proof.lean` with the real proofs). The **M0 tier** is a cycle-level scheduler semantics over 4-state values: `Logic := 0/1/x/z`, LSB-first vectors `LVec`, verified operator tables (whole-vector X-collapse for arithmetic per LRM §11.4.3, ternary bitwise-merge per §11.4.11, `if-with-x` takes else), and one `cycleStep` = inputs $\rightarrow$ combinational settle $\rightarrow \sigma$ -ordered edge phase (blocking assigns immediate, nonblocking to a queue) $\rightarrow$ NBA commit $\rightarrow$ settle. The judgment family:

<code>d / stim $\Downarrow_\sigma$ tr</code>	design under stimulus and schedule yields trace (fuel hidden by monotonicity)
<code>d $\models$ P</code>	$\forall \sigma$ stim tr: trace property — for <i>all</i> legal schedules
<code>Sv.Deterministic d</code>	all schedules agree (race-freedom)
<code>d $\sqsubseteq$ @clk[from rst] m</code>	cycle refinement of a Lean golden model, from first sampled reset
<code>#sv_check d [stim] shows s = [..]</code>	concrete interpreter run, checked at elaboration

Proved so far: all six gallery designs (counter golden-model refinement; the `race_blk nondeterminism theorem` — two legal schedules, two outcomes, a statement no simulator can make; `swap_nba` schedule-independence; adder with whole-vector X-collapse; `xsel` X-optimism; a T-flip-flop refinement), each with realizability/non-vacuity checks, dual-simulator differential harness (`harness/sv/diff_test.py -sim {auto,xrun,iverilog}`) green in cloud CI. A hard lesson worth internalizing: this codebase treats *vacuity* as the enemy. Every behavior/safety theorem ships with a paired realizability witness; audits attack hypothesis sets with kernel evidence. Twice, plausible-looking theorem sets were found vacuous or spec-baked by adversarial audit; the pairing rule and a “spec-in-behavior” lens are now standing law.

3 The goal: CV32E40P, file by file, up to the ISA

The north star is proving **every RTL file of the OpenHW CV32E40P** (the shipped ETH/OpenHW RISC-V RV32IMC core, 27 files) correct — leaf modules with real local specs, pipeline stages as *contracts*, composed at the summit into $core \sqsubseteq isaStep$. Two **design laws** (user-mandated) distinguish our approach from all prior BMC-style verification of this core: *parameters stay symbolic* — a parameterized module extracts to a **Design**-valued *function* of its parameters, so theorems quantify $\forall C_WIDTH$ (crown jewels: **ff_one** $\forall LEN$, the serial divider $\forall width$, FIFO $\forall depth$, register file $\forall addr$) — and *generate stays structural* — a **generate for** is one node, an indexed family proved by induction, never unrolled. Elaboration ($instantiate : Design \rightarrow Params \rightarrow FlatDesign$) is a *Lean function* proofs reason about, not an extractor pass we trust.

Where we are (all committed): the **symbolic extractor** (sv-0.2, -top multi-file mode; name resolution only) and the **census** (docs/cv32e40p-coverage.md): 27/27 files extract; 11,446 nodes = 10,005 concrete + 867 symbolic + **574 unsupported** — 100% genuine semantic-tier work. The tier ladder: **T-select** (bit/part-select, select targets, reductions, replication, **\$signed**, **unique case ... inside** — note: the core contains *no casez*) clears **ff_one**/popcnt; **T-reset** (async active-low reset — used by *every* clocked module; the current [from rst] machinery fits none of them) unlocks FIFO and register file; then T-array (unpacked arrays), T-hier (instantiation). The **spec gallery** (docs/cv32e40p-spec-surface.md) drafts all ten target statements against the actual RTL and derives the new surface vocabulary: rely-guarantee contracts $\models^c$ over port traces; *ghost state* (the controller’s instruction ledger — exactly-once retire-or-squash; ports carry no instruction identity, so auxiliary spec state is minted from handshakes; principled by Abadi–Lamport history variables, with Sawada–Hunt’s MAETT as direct prior art); environment oracles (β for the OBI bus); spec-side DSLs as data (CSR behaviors, decode table, abstract memory); and the honest RTL findings that would have broken naive specs (write port B beats A; ALU_ADD routes through the shifter; **ff_one** is broken at $LEN=1$; the divider’s operands arrive pre-swapped).

4 The ISA model and the projection architecture

The three hardest files (controller, CSR file, LSU) all lean on the privileged spec, and four spec-side artifacts — decode table, ALU op semantics, div/rem conventions (RISC-V never traps on division: $x/0 = -1$, $\text{rem } x\ 0 = x$, $\text{MinInt}/-1 = \text{MinInt}$), CSR/WARL behaviors — are *projections of one model*. Writing them independently means four hand transcriptions of the same document and hundreds of bridging lemmas at composition time. So we build **one** Lean RV32IMC+privileged model (**LeanModels/Rv/****) and every module spec consumes its fragments; composition then holds by construction. A literature review (docs/litreview/) settled the key call: **build, don’t import**. The Sail→Lean backend is real and funded but emits $\sim 175K$ lines of structure-preserving (hence illegible) Lean; instead we hand-write in a *semantic-bucketed* AST (~ 5 orthogonal constructor families, not opcode-per-constructor — avoids quadratic **no_confusion**; confirmed by LNSym and riscv-coq practice), transliterate the fiddly tables from sail-riscv’s readable source (BSD-2, attributed), and use the **Sail emulator as differential oracle** — the Xcelium pattern applied to the ISA.

5 sv_vcgen: the proof engine

The Python lane’s endgame was **py_vcgen**: a verification-condition generator over the deep AST — flow-aware Hoare triples proved sound per constructor, a walker tactic that turns a goal into VCs, the user supplying only invariants and measures, residuals presented as named-atom arithmetic. It reproved a vendored **python-rsa** function at 143 lines (from 330) and proved nested-loop/break programs the old tactics could not; cold agents with no project context have since proved CPython’s **bisect_left** with it. **sv_vcgen** transplants this validated architecture to hardware with seven deltas (banked spec in project memory): **(1)** two closer tiers — a symbolic-width **LVec** lemma pack for $\forall$ -parameter goals, **bv_decide**-style certificate checking only at concrete instances; **(2)** *per-state invariants* for FSMs (**sv_vcgen** (inv **IDLE** := ...) (inv **DIVIDE** := ...) (dec := ...)), preservation VCs forked per (state, case-arm) with legible tags; **(3)** a **case inside** rule with 4-state matching; **(4)** a transaction judgment $\models^{tx}$ (request $\rightsquigarrow$ response within a deadline that is a *function of the module’s parameters*) for multi-cycle units like the divider; **(5)** generate-family VCs closed by induction; **(6)** the *discipline metatheorem* (after Choi et al., OOPSLA’25): a syntactic NBA-discipline check that discharges $\forall \sigma$ once for the whole single-clock core; **(7)** module contracts at instantiation boundaries, composed by McMillan’s “constrains” rule ($p \triangleright q$, CHARME’99) — the register boundary supplies exactly the $t \rightarrow t+1$ offset that makes circular rely-guarantee

sound. Acceptance benchmarks are fixed in advance: `ff_one` and `popcnt` first ($\forall$ -width combinational), the serial divider as the boss ($\forall$ width `tx` div/rem with the RISC-V conventions), and the existing ~ 300 -line hand-built counter/toggle proofs must collapse to clause form with statements byte-preserved.

6 Where things stand and how we work

Running now: Phase 2 (Lean ingestion of symbolic envelopes + the T-select/case-inside/T-reset tier, differential rows first; acceptance: the actual `ff_one`/`popcnt`/divider RTL executing under our interpreter against simulator traces) and the ISA model build (with Sail-emulator validation). **Next:** fire `sv_vcgen` when Phase 2 lands; then the module ladder — leaves, memory-ish modules, decoder, FSMs — then contracts, then the summit. **House workflow you should adopt:** gallery-first (statements agreed before machinery); differential rows before semantics; existing theorem statements are byte-preserved through refactors; every new theorem gets `#print axioms` pinned (standard axioms only — no `sorry`, `native_decide`, or custom axioms, ever); adversarial audits with kernel-checkable evidence before landing (they have caught real vacuity twice — treat them as a gift); `bash tools/ci.sh` is the full local gate and cloud CI mirrors it. **Reading order:** this brief; `README.md`; `docs/sv-design-m0.md` and `docs/sv-spec-surface.md` (lane contract and gallery); `docs/cv32e40p-coverage.md` and `docs/cv32e40p-spec-surface.md` (the program); `docs/litreview/SYNTHESIS.md` (why each big design call went the way it did); then `LeanModels/Sv/Semantics.lean` and one example pair (`Examples/system-verilog/counter/`). Welcome aboard — the summit theorem ($cv32e40p \sqsubseteq isaStep$, on the shipped core, through our own front door, at symbolic width) has not been done by anyone; every rung toward it is a publishable artifact on its own.